



FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL ENGINEERING

PROJECT REPORT

SPI SLAVE WITH SINGLE-PORT RAM

DESIGN AND IMPLEMENTATION USING FPGA



PREPARED BY

Farha Fouad Abdelhamid
Mohamed Mahmoud Elaaser



DATE

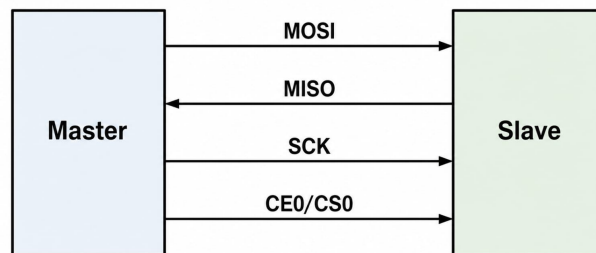
MAY 2025

1 Design Blocks

1.1 SPI Master

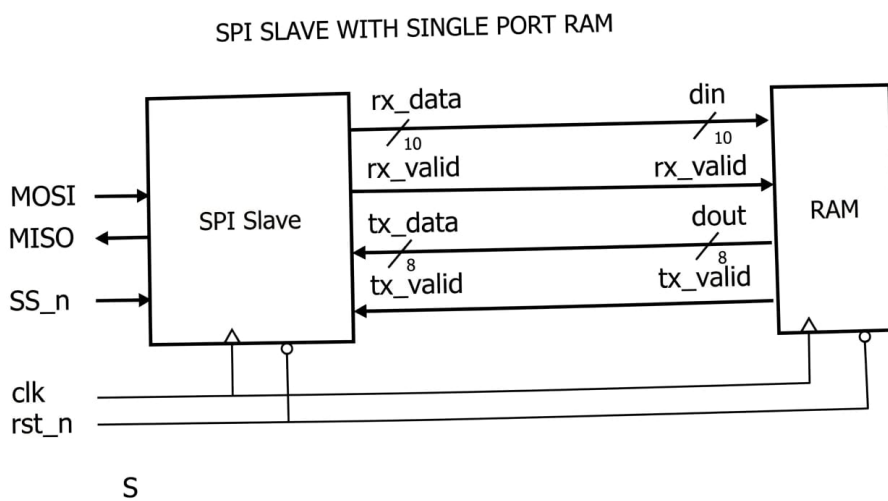
The SPI Master controls the communication process by generating the serial clock (SCLK) and the chip select signal (SS_n). It transmits commands, addresses, and data to the SPI Slave through the MOSI line and receives data through the MISO line.

1. MOSI : Master-out-slave-in
2. MISO : Master-in-slave-out
3. SCLK : CLOCK
4. SS : slave select



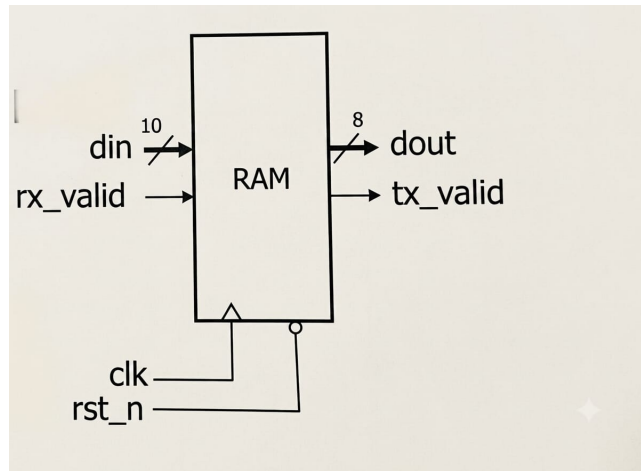
1.2 SPI Slave

The SPI Slave receives serial data from the SPI Master through the MOSI line. It decodes the received command, controls the read/write operations, and communicates with the Single-Port RAM. During read operations, it transmits the requested data through the MISO line.



1.3 Single-Port RAM

The Single-Port RAM stores the transmitted data using a single access port. It performs either a read or a write operation at a given address based on the control signals generated by the SPI Slave.



1.4 Wrapped Module

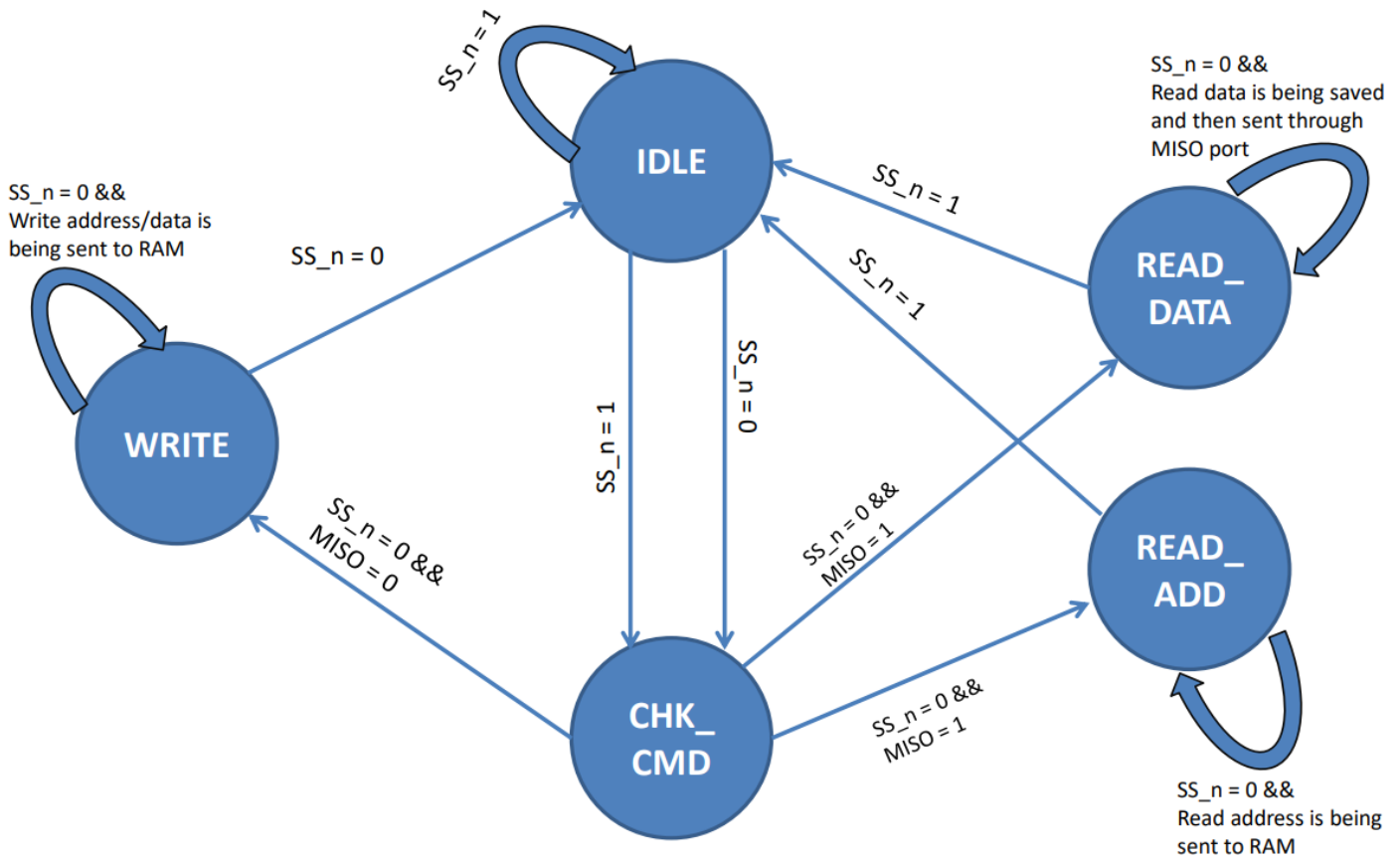
The Wrapped module is the top-level module that integrates the SPI Slave and the Single-Port RAM. It connects all internal signals and coordinates the communication between the SPI interface and the memory, providing a complete SPI memory system.

RAM Interface Signals

- **din** (Input, 10 bits): Data Input
- **clk** (Input, 1 bit): Clock
- **rst_n** (Input, 1 bit): Active low asynchronous reset
- **rx_valid** (Input, 1 bit): If HIGH: accept `din[7:0]` to save the write/read address internally or write a memory word depending on the most significant 2 bits `din[9:8]`
- **dout** (Output, 8 bits): Data Output
- **tx_valid** (Output, 1 bit): Whenever the command is memory read the `tx_valid` should be HIGH

Parameters

- **MEM-DEPTH** , Default : 256
- **ADDR-size** , Default : 8



2 Finite State Machine (FSM)

2.1 SPI Slave FSM

The SPI Slave is controlled by a Finite State Machine (FSM) consisting of five states:

- **IDLE:** Default state. The slave waits until $SS_n = 0$.
- **CHK_CMD:** The received command is checked to determine the required operation:
 - Write Command → WRITE
 - Read Command → READ_ADD
- **WRITE:** The slave receives the address and data through SPI and writes the data into the Single-Port RAM.
- **READ_ADD:** The slave receives the memory address to be read.
- **READ_DATA:** The slave reads the requested data from RAM and transmits it serially through the MISO line.

After completing any operation or when $SS_n = 1$, the FSM returns to the **IDLE** state.

RAM Write Command – Write Address [00]

1. Master will start the write command by sending the write address value, $rx_data[9 : 8] = din[9 : 8] = 2'b00$

2. $SS_n = 0$ to tell the SPI Slave that the master will begin communication
3. SPI Slave check the first received bit on MOSI port '0' which is a control bit to let the slave determine which operation will take place "write in this case". SPI Slave then expects to receive 10 more bits, the first 2 bits are "00" on two clock cycles and then the $wr_address$ will be sent on 8 more clock cycles
4. Now the data is converted from serial "MOSI" to parallel after writing the $rx_data[9 : 0]$ bus. $rx_data[9]$ holds the first bit received from the MOSI port and $rx_data[0]$ holds the last bit received.
5. rx_valid will be HIGH to inform the RAM that it should expect data on din bus
6. din takes the value of rx_data
7. RAM checks on $din[9 : 8]$ and find that they hold "00"
8. RAM stores $din[7 : 0]$ in the internal write address bus
9. $SS_n = 1$ to end communication from Master side

RAM Write Command – Write Data [01]

1. Master will continue the write command by sending the write data value, $rx_data[9 : 8] = din[9 : 8] = 2'b01$
2. $SS_n = 0$ to tell the SPI Slave that the master will begin communication
3. SPI Slave check the first received bit on MOSI port '0' which is a control bit to let the slave determine which operation will take place "write in this case". SPI Slave then expects to receive 10 more bits, the first 2 bits are "01" on two clock cycles and then the wr_data will be sent on 8 more clock cycles
4. Now the data is converted from serial "MOSI" to parallel after writing the $rx_data[9 : 0]$ bus. $rx_data[9]$ holds the first bit received from the MOSI port and $rx_data[0]$ holds the last bit received.
5. rx_valid will be HIGH to inform the RAM that it should expect data on din bus
6. din takes the value of rx_data
7. RAM checks on $din[9 : 8]$ and find that they hold "01"
8. RAM stores $din[7 : 0]$ in the RAM with $wr_address$ previously held
9. $SS_n = 1$ to end communication from Master side

RAM Read Command – Read Address [01]

1. Master will start the read command by sending the read address value, $rx_data[9 : 8] = din[9 : 8] = 2'b10$
2. $SS_n = 0$ to tell the SPI Slave that the master will begin communication

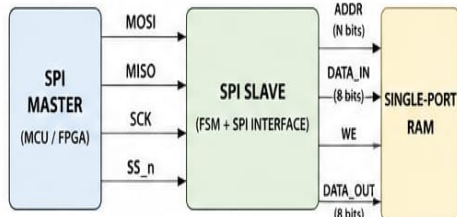
3. SPI Slave check the first received bit on MOSI port '1' which is a control bit to let the slave determine which operation will take place "read in this case". SPI Slave then expects to receive 10 more bits, the first 2 bits are "10" on two clock cycles and then the *rd_address* will be sent on 8 more clock cycles
4. Now the data is converted from serial "MOSI" to parallel after writing the *rx_data*[9 : 0] bus. *rx_data*[9] holds the first bit received from the MOSI port and *rx_data*[0] holds the last bit received.
5. *rx_valid* will be HIGH to inform the RAM that it should expect data on din bus
6. *din* takes the value of *rx_data*
7. RAM checks on *din*[9 : 8] and find that they hold "10"
8. RAM stores *din*[7 : 0] in the internal read address bus
9. *SS_n* = 1 to end communication from Master side

RAM Read Command – Read Data [01]

1. Master will continue the read command by sending the read data command, *rx_data*[9 : 8] = *din*[9 : 8] = 2'b11
2. *SS_n* = 0 to tell the SPI Slave that the master will begin communication
3. SPI Slave check the first received bit on MOSI port '1' which is a control bit to let the slave determine which operation will take place "read in this case". SPI Slave then expects to receive 10 more bits, the first 2 bits are "11" on two clock cycles and then dummy data will be sent and ignored since the master is waiting for the data to be sent from slave side
4. Now the data is converted from serial "MOSI" to parallel after writing the *rx_data*[9 : 0] bus. *rx_data*[9] holds the first bit received from the MOSI port and *rx_data*[0] holds the last bit received.
5. *din* takes the value of *rx_data*
6. RAM reads *din*[9 : 8] and find that they hold "11"
7. RAM will read from the memory with *rd_address* previously held
8. RAM will assert *tx_valid* to inform slave that data out is ready
9. Slave reads *tx_data* and convert it into serial out data on MISO port starting with the most significant bit.
10. *SS_n* = 1, Master ends communication after receiving data "8 clock cycles"

SPI SLAVE WITH SINGLE-PORT RAM – COMPLETE OVERVIEW

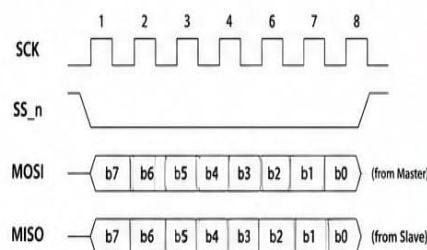
1) BLOCK DIAGRAM



2) SIGNAL DESCRIPTION

Signal	Dir.	Description
MOSI	In	Master Out Slave In (serial data from Master)
MISO	Out	Master In Slave Out (serial data to Master)
SCK	In	Serial Clock from Master
SS_n	In	Chip Select (Active Low)
ADDR	Out	Address bus to RAM (N bits)
DATA_IN	Out	Data bus to RAM (8 bits)
DATA_OUT	In	Data bus from RAM (8 bits)
WE	Out	Write Enable for RAM (1 bit)

3) SPI COMMUNICATION (Mode 0)



Data is sampled on the rising edge of SCK (Mode 0) and shifted out on the falling edge.

4) COMMAND FORMAT (FROM MASTER)



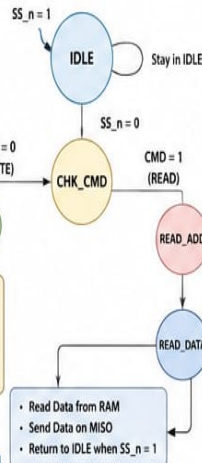
CMD	Operation
0	WRITE (Address + Data)
1	READ (Address → Data)

WRITE Example : [0 | Address | Data]

Master sends Address and Data → Slave writes Data to RAM

READ Example : [1 | Address]

Master sends Address → Slave sends Data through MISO

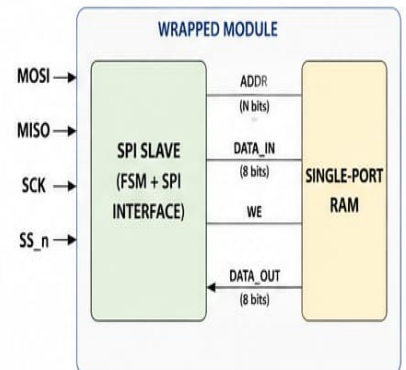


6) SINGLE-PORT RAM

Address (N bits)	Data (8 bits)	RAM I/O
0x00	8'h3A	- One port only - At a time: Either Read OR Write - Controlled by WE signal
0x01	8'h7F	
0x02	8'hC1	
:	:	
0xFE	8'h55	
0xFF	8'hAA	WE = 1 → Write WE = 0 → Read

WE = 1 : Write operation (Data is written to RAM)
WE = 0 : Read operation (Data is read from RAM)

7) WRAPPED MODULE (TOP LEVEL)



8) DATA TRANSFER SEQUENCE

WRITE SEQUENCE (CMD = 0)			
CMD (0)	ADDRESS	DATA	
1 bit	N bits	8 bits	

- 1) Master Selects Slave (SS_n = 0)
- 2) Send CMD + Address + Data
- 3) Slave Writes Data to RAM
- 4) SS_n = 1 (End)

READ SEQUENCE (CMD = 1)			
CMD (1)	ADDRESS	DUMMY (0x00)	DATA FROM SLAVE
1 bit	N bits	8 bits	8 bits

- 1) Master Selects Slave (SS_n = 0)
- 2) Send CMD + Address
- 3) Master sends dummy byte to generate clocks
- 4) Slave sends Data on MISO
- 5) SS_n = 1 (End)

9) KEY FEATURES

- ✓ Full duplex SPI communication
- ✓ Supports Read and Write operations
- ✓ Single-Port RAM for data storage
- ✓ Simple and efficient FSM control
- ✓ Mode 0 SPI (CPOL = 0, CPHA = 0)
- ✓ Suitable for MCU / FPGA communication



10) APPLICATIONS

- Data Logging Systems
- Sensor Data Storage
- Register Configuration
- Memory Expansion
- Communication between FPGA and MCU



3 Code Discription

3.1 SPI Slave Module Description

The SPI Slave module acts as an interface between the external SPI Master and the RAM. It converts serial data received on the MOSI line into a 10-bit parallel word and sends it to the RAM. During read operations, it converts the 8-bit parallel data received from the RAM into serial data and transmits it through the MISO line.

- **FSM States**

- **IDLE**: Waits until the slave is selected.
- **CHK_CMD**: Decodes the received command.
- **WRITE**: Receives a 10-bit write command.
- **READ_ADD**: Receives the read address.
- **READ_DATA**: Sends the requested data to the master.

- **State Register**

- **cs** stores the current state.
- **ns** stores the next state.
- The current state is updated on every rising clock edge.

- **Command Detection**

- If **SS_n** = 0, communication starts.
- **MOSI** = 0 selects the Write operation.
- **MOSI** = 1 selects the Read operation.
- The flag **received_add** determines whether the received bits represent a read address or read data.

- **Serial-to-Parallel Conversion**

- Incoming bits on **MOSI** are stored sequentially in **rx_data**.
- The counter **s_p_counter** tracks the received bits.
- After receiving all 10 bits, **rx_valid** is asserted to notify the RAM.

- **Parallel-to-Serial Conversion**

- When **tx_valid** is asserted, the SPI Slave transmits **tx_data** bit-by-bit through **MISO**.
- The counter **p_s_counter** controls the transmitted bit position.

- **Read and Write Operations**

1. During a write operation, the SPI Slave receives a 10-bit command and forwards it to the RAM.
2. During a read operation, it first receives the read address, waits for the RAM to provide the requested data, then serializes and transmits the data to the SPI Master.

3.2 RAM Module Description

The RAM module is responsible for storing and retrieving data according to the command received from the SPI slave. The input `din` is 10 bits wide, where the upper two bits represent the operation command and the lower eight bits represent either an address or data.

- **Module Declaration**

- `module RAM(...)` defines the RAM block.
- `MEM_DEPTH = 256` specifies that the memory contains 256 locations.
- `ADDR_SIZE = 8` indicates that each address is 8 bits wide.

- **Inputs and Outputs**

- `din[9:0]`: 10-bit command from the SPI slave.
- `clk`: System clock.
- `rst_n`: Active-low reset.
- `rx_valid`: Indicates that the received command is valid.
- `dout[7:0]`: Data read from memory.
- `tx_valid`: Indicates that the output data is valid.

- **Memory Declaration**

- `reg [7:0] mem [255:0]` creates a memory of 256 locations.
- Each location stores one byte (8 bits).

- **Address Registers**

- `wr_add`: Stores the write address.
- `rd_add`: Stores the read address.

- **Clocked Process**

- The RAM operates on the rising edge of the clock using `always @(posedge clk)`.

- **Reset Operation**

- When `rst_n = 0`, all control registers are cleared.
- `dout`, `tx_valid`, `wr_add`, and `rd_add` are initialized to zero.

- **Command Decoder**

- 00: Store the write address in `wr_add`.
- 01: Write `din[7:0]` into `mem[w_r_add]`.
- 10: Store the read address in `rd_add`.
- 11: Read `mem[r_d_add]` into `dout` and set `tx_valid = 1`.

- **Write Operation**

1. Receive command 00 with the target address.
2. Receive command 01 with the data.

3. Store the data in the selected memory location.

- **Read Operation**

1. Receive command 10 with the target address.
2. Receive command 11.
3. Read the stored value from memory.
4. Output the value on `dout` and assert `tx_valid`.

3.3 SPI Wrapper Module Description

The SPI Wrapper is the top-level module that connects the SPI Slave and the RAM modules. It does not perform any data processing; instead, it provides the communication interface between both modules.

- **Module Parameters**

- `MEM_DEPTH` = 256: Defines the RAM size.
- `ADDR_SIZE` = 8: Defines the address width.
- The FSM state parameters are passed to the SPI Slave module.

- **Internal Signals**

- `rx_data`: 10-bit data transferred from the SPI Slave to the RAM.
- `rx_valid`: Indicates that the received command is valid.
- `tx_data`: 8-bit data returned from the RAM.
- `tx_valid`: Indicates that the output data from the RAM is valid.

- **SPI Slave Instantiation**

- Receives serial data from the SPI Master through `MOSI`.
- Converts the serial data into a 10-bit parallel word.
- Sends `rx_data` and `rx_valid` to the RAM.
- Transmits the RAM output through the `MISO` line.

- **RAM Instantiation**

- Receives commands from the SPI Slave.
- Executes read or write operations.
- Returns the read data through `tx_data`.
- Generates `tx_valid` when the requested data is ready.

- **Overall Function**

- The SPI Wrapper integrates both modules to implement the complete SPI memory interface, allowing the SPI Master to read from and write to the RAM.